

Technical Baseline & Project Standards

All 10 project topics are designed with a **balanced, production-grade difficulty level**. Every project simulates a real-world business or interactive domain system and requires robust Object-Oriented Programming (OOP) architecture.

Feature Area	Mandatory Technical Expectation across ALL Topics
Object-Oriented Architecture	Must use <code>class</code> hierarchy exclusively (no <code>struct</code>). Full application of Encapsulation, Inheritance, Polymorphism, and Abstraction.
Data Structures & STL	Effective use of C++ Standard Template Library (<code>std::vector</code> , <code>std::map</code> , <code>std::string</code> , <code>std::stack</code> , <code>std::algorithm</code>).
File Persistence	Save and load system state to/from files (<code>std::ifstream</code> , <code>std::ofstream</code> , CSV/Text format).
User Experience & CLI	Interactive, formatted Console Interface with menu navigation, input validation, and clear status feedback.
Version Control	Managed via Git with individual commit histories and Pull Requests (PRs).

Mandatory Project Requirements

1. Git Workflow & Collaboration Rules

- **Git Version Control:** Every project must be managed with Git and hosted on a cloud platform (GitHub or GitLab).
- **Individual Commit History:** For pair projects, **each student must contribute code through their own account and commit regularly**. The instructor will track and grade individual contributions by analyzing the Git commit history. Projects where only one student commits will be penalized.

- **Pull Request (PR) Workflow:** Direct pushing or merging code into `main` or `master` is **strictly prohibited**. All feature additions or updates must be developed on separate branches and merged via **Pull Requests (PRs)**.

2. Object-Oriented Programming (OOP) Core Standards

- **Mandatory `class` Architecture:** All projects **must use `class`** design. Procedural scripts or `struct`-only data structures without encapsulation are not accepted.
- **OOP Pillars Implementation:**
 - **Encapsulation:** Data members must be `private` or `protected`, exposed only via getter/setter methods and behavior-driven public APIs.
 - **Inheritance & Polymorphism:** Design base classes with `virtual` functions and derived classes overriding behavior.
 - **Abstraction & Modularity:** Clean separation of business logic, domain models, and storage handlers into dedicated classes.

General Grading Criteria

Criteria	Weight	Description
Runs & Stability	25%	Program compiles cleanly, handles edge cases, runs without crashes
Business Requirements	25%	All core domain features implemented completely and accurately
OOP Architecture & Quality	25%	Effective <code>class</code> design, encapsulation, inheritance, polymorphism, clean code
Git Workflow & History	15%	Active individual commit history, PR workflow followed, clean branch management
Report & Demo Presentation	10%	Clear documentation, smooth live demo, ability to explain code logic

DETAILED TOPICS (10 OOP PROJECTS)

Topic 1: Note System

Description: Build an enterprise-grade console note and knowledge management system. Users can organize different types of notes (Text, Checklist, Code Snippet), categorize them, search using keywords/tags, and persist data to files.

Key Business Requirements:

#	Business Requirement	Technical Implementation
1	Polymorphic Note Types (<code>TextNote</code> , <code>ChecklistNote</code> , <code>CodeNote</code>)	Inheritance from abstract <code>Note</code> class, <code>virtual</code> display methods
2	CRUD Notes (Create, Read, Update, Delete)	<code>NoteManager</code> class managing <code>std::vector<Note*></code>
3	Category & Tagging System	<code>std::map<std::string, std::vector<Note*>></code> for category grouping
4	Keyword & Tag Search Engine	String parsing, case-insensitive substring search
5	Lock & Security Protection for Private Notes	Password protection using <code>class EncryptedNote</code>
6	Import & Export Persistence	Save/Load formatted text/CSV files via <code>StorageService</code> class
7	Summary Analytics & Statistics	Word counts, note breakdown by category, creation timeline

Topic 2: Personal Financial Management System

Description: Build a comprehensive financial and budget management application for students and freelancers. Track income, expenses, multi-account balances, recurring bills, and budget limit alerts with visual reports.

Key Business Requirements:

#	Business Requirement	Technical Implementation
1	Transaction Types (Income, Expense, RecurringExpense)	BaseTransaction class with derived specific classes
2	Multi-Account Balances (Cash, Bank, E-Wallet)	Account class managing individual balances
3	Category Budget Limits & Over-budget Alerts	Budget class checking threshold conditions
4	Recurring Transaction Scheduler Simulation	Auto-calculate monthly fixed expenses
5	Filtering & Searching (by Date Range, Category, Account)	FinanceManager query methods
6	ASCII Chart Visualization	Bar chart generation (█) for monthly spending comparison
7	CSV Export & Data Load	File stream parsing viaFinanceStorage class

Topic 3: Interactive Multi-Language Vocabulary System

Description: Build an intelligent vocabulary learning and dictionary system. Supports word management across multiple word types, synonym mapping, flashcard quiz modes with Leitner spaced repetition, and study analytics.

Key Business Requirements:

#	Business Requirement	Technical Implementation
1	Word Categorization (GeneralWord, TechnicalTerm, Idiom)	Derived classes from abstractWord class
2	Dual-Direction Lookup (English ↔ Vietnamese)	std::map<std::string, Word*> fast indexed lookup
3	Synonyms, Antonyms, & Example Sentences	Vector attributes insideWord encapsulation

#	Business Requirement	Technical Implementation
4	Spaced Repetition Study Engine (Leitner Box System)	Mastery level tracking (1 to 5) per word
5	Interactive Self-Assessment & Auto-Graded Quiz	Quiz session class with score calculations
6	File Import/Export (Dictionary Database Persistence)	Structured text file loader/writer
7	Progress Dashboard & Performance Metrics	Accuracy percentage, weak words identification

Topic 4: Smart Library & Resource Management System

Description: Build a digital library and media resource management system. Manage diverse library items (Physical Books, E-Books, Academic Journals), member subscriptions, borrowing policies, late return fines, and analytics.

Key Business Requirements:

#	Business Requirement	Technical Implementation
1	Polymorphic Library Resources (Book , EBook , Journal)	Abstract LibraryResource base class
2	Member Subscription Types (StudentMember , FacultyMember)	Abstract Member class with different borrow limits & fine rates
3	Borrow & Return Loan Processing Workflow	Validation of copy availability and member limits
4	Fine Calculation Engine for Overdue Loans	Polymorphic calculateFine(int overdueDays) method
5	Resource Reservation & Queue System	Member waiting queue per unavailable resource
6	Search Engine (by Title, Author, ISBN, Genre)	Custom predicates with std::find_if

#	Business Requirement	Technical Implementation
7	Data Persistence & Inventory Report Export	Complete state load/save to files

Topic 5: Interactive Flashcard & Examination Engine

Description: Build an interactive flashcard learning and exam simulation system. Supports diverse question formats (Multiple Choice, True/False, Fill-in-the-Blank, Code Output), custom test deck creation, timed exam modes, and detailed performance breakdown.

Key Business Requirements:

#	Business Requirement	Technical Implementation
1	Polymorphic Question Types (MultipleChoice, FillBlank, CodeOutput)	Derived classes from abstractQuestion class
2	Deck & Subject Management	Deck class holding collections of questions
3	Timed Exam Simulation Mode	Countdown timer simulation, score calculation
4	Adaptive Practice Mode	Re-test missed questions based on historical accuracy
5	Detailed Result Breakdown & Analytics	Percentage, time per question, topic weakness analysis
6	Question Bank File Persistence	Save/Load question decks from formatted text/CSV files

Topic 6: Cinema Multiplex Ticket & Seat Reservation System

Description: Build a multi-screen cinema ticketing and reservation platform. Manage movies, showtimes, dynamic seating layouts (Standard, VIP, Couple seats), ticket pricing rules, promotional discounts, and daily revenue reporting.

Key Business Requirements:

#	Business Requirement	Technical Implementation
1	Dynamic Seat Types (StandardSeat , VIPSeat , CoupleSeat)	Derived from abstract Seat class with pricing polymorphism
2	Interactive ASCII Seat Map Display	2D array representation of room layout
3	Movie Showtime Scheduling System	Showtime class connecting Movie, Room, and Seat Map
4	Booking & Seat Hold Workflow	Validation of availability, multi-seat reservation
5	Pricing Strategy & Discount Rules	Weekend surcharges, student discounts using Strategy pattern
6	Sales Analytics & Daily Revenue Summary	Aggregation by movie, time slot, seat tier
7	Booking Receipt Export & State Persistence	Write electronic ticket files, save room states

Topic 7: Strategy Board Game Engine (Gomoku / Connect-5)

Description: Build a complete 2-player Gomoku (Five-in-a-row) board game engine. Supports customizable board dimensions, player vs player and player vs AI modes, move history with Undo/Redo capabilities, game saving, and match replay.

Key Business Requirements:

#	Business Requirement	Technical Implementation
1	Polymorphic Player System (HumanPlayer , AIPlayer)	Derived classes from abstract Player class
2	Customizable Grid Board Management	Board class encapsulating 2D grid logic

#	Business Requirement	Technical Implementation
3	Directional Win-Checking Algorithm	4-axis line search (Horizontal, Vertical, 2 Diagonals)
4	Undo / Redo System	Dual <code>std::stack<Move></code> for move tracking
5	Game State Persistence (Save / Load Game)	Serialization of board state and turn history to file
6	Match Replay Engine	Step-by-step playback of recorded match moves
7	Leaderboard & Player Statistics	Win/loss ratio tracking saved in file

Topic 8: Console Digital Media & Music Player System

Description: Build a digital media library and music player management system. Supports multiple media types (Songs, Podcasts, Audiobooks), custom playlist creation, playback state machine simulations (Play, Pause, Shuffle, Repeat), and listening statistics.

Key Business Requirements:

#	Business Requirement	Technical Implementation
1	Polymorphic Media Types (<code>Song</code> , <code>Podcast</code> , <code>Audiobook</code>)	Abstract <code>MediaContent</code> base class
2	Media Library Management (Add, Remove, Search, Filter)	<code>MediaLibrary</code> class storing <code>std::vector<MediaContent*></code>
3	Playlist Management (Create, Edit, Reorder, Delete)	<code>Playlist</code> class maintaining pointers to media
4	Playback State Machine & Modes (Sequential, Shuffle, Repeat)	Enum state control and random shuffle algorithm
5	Interactive Player Console Interface	Progress bar simulation ( <code>02:15/04:00</code>)

#	Business Requirement	Technical Implementation
6	Play Count Statistics & Most Listened Charts	Sorting media items by frequency of playback
7	Library & Playlist State Persistence	Save/Load library catalog and playlists to file

Topic 9: Interactive Text RPG & Storytelling Engine

Description: Build a text-based RPG and decision-tree storytelling game engine. Features hero class selection (Warrior, Mage, Ranger), monster battles, inventory/equipment management, branching storyline choices, and save points.

Key Business Requirements:

#	Business Requirement	Technical Implementation
1	Hero Classes (Warrior, Mage, Ranger)	AbstractHero class with polymorphic skills
2	Enemy Types (Minion, BossMonster)	DerivedEnemy classes with custom AI attacks
3	Turn-Based Tactical Combat Engine	Attack calculations, defense mitigation, skill cooldowns
4	Inventory & Item Management (HealthPotion, Weapon, Armor)	Item class hierarchy and player inventory
5	Branching Narrative Graph (StoryNode)	Directed graph or tree for story choices and outcomes
6	Character Leveling & Stat Progression	EXP tracking, stat allocation upon level up
7	Game Save & Load Checkpoints	Complete game state serialization to file

Topic 10: Digital Media Processing Tool (Image PPM / Text Analyst Engine)

Description: Build a digital media processor for PPM image manipulation (Portable Pixmap format P3) OR an advanced text document analyzer. Students select **Track A** or **Track B**.

Track A: PPM Image Processing Engine

Key Business Requirements:

#	Business Requirement	Technical Implementation
1	Polymorphic Image Filters (GrayscaleFilter , BrightnessFilter , InvertFilter)	Derived classes from abstract ImageFilter class
2	ColorPixel Class Encapsulation	RGB bounds checking (0-255) and pixel operations
3	PPMImage File Parsing & Writing	Direct file I/O for PPM P3 format
4	Geometry Manipulations	Horizontal/Vertical Flip, 90-degree Rotation
5	Region Cropping & Image Resizing	Matrix transformation algorithms
6	Spatial Filter Convolutions (Blur, Sharpen)	3x3 kernel matrix calculations

Track B: Text Document Analyst Engine

Key Business Requirements:

#	Business Requirement	Technical Implementation
1	Document Analysis (WordCount , FrequencyMap , N-Gram)	TextAnalyzer class
2	Pattern Extraction (EmailExtractor , URLExtractor)	Derived from abstract PatternExtractor class
3	Find & Replace Engine	Substring replacement, case-sensitivity flags
4	File Diff Engine	Compare 2 text files and highlight line differences
5	File Merger & Line Sorting	File I/O, std::sort with custom comparators

#	Business Requirement	Technical Implementation
6	Analytics Report Exporter	Export structured summary to text file

Detailed Grading Rubric

Technical Evaluation Matrix (10 Points Scale)

Score Range	OOP Architecture	Git & Collaboration	Business Logic & Code Quality	Demo & Defense
9.0 – 10.0	Exceptional class design. Proper inheritance, virtual functions, abstract base classes, complete encapsulation.	Regular commits from both members, clear PR history, proper feature branching.	All business requirements met, handles edge cases, zero compiler warnings, robust File I/O.	Fluent live demo, answers all instructor code-defense questions confidently.
7.5 – 8.9	Good class usage with encapsulation and inheritance. Minor design flaws.	Both members commit, PRs used correctly.	85%+ features work as expected, file persistence functional.	Good presentation, answers most defense questions accurately.
5.0 – 7.4	Basic class implementation. Weak encapsulation or limited polymorphism.	Irregular commits or single member dominant. Few PRs.	Core features work, minor bugs or edge case crashes.	Basic demo, struggles with deeper questions about code logic.
< 5.0	Fails OOP requirements (procedural, struct -only).	Direct commits to main , single student commit, or no Git repo.	Major features missing or crashes frequently.	Cannot explain code logic → potential

Score Range	OOP Architecture	Git & Collaboration	Business Logic & Code Quality	Demo & Defense
				plagiarism flag.